Week 18 – Advanced Topics & Edge Cases

■ Objectives

• Deepen recursion on tree-like problems (safe base cases).
• Handle edge cases: empty inputs, large numbers, weird strings.
• Intro to file I/O for larger inputs (optional).

Recursion – Tree Sum

```
# Tree represented as dict: node -> [children]
tree = {1:[2,3], 2:[4], 3:[], 4:[]}
values = {1:5, 2:3, 3:2, 4:4}
def tree_sum(u):
    total = values[u]
    for v in tree[u]:
        total += tree_sum(v)
    return total
print(tree_sum(1))  # 14
```

Edge Cases – Defensive Programming

• Check for empty lists/strings before indexing.
• Guard divisions by zero; clamp indices to ranges.
• Validate inputs; use `.strip()` and `.split()` carefully.

File I/O Basics (Optional)

```
# Read numbers from a file and print their sum
# with open('input.txt') as f:
#     nums = [int(x) for x in f.read().split()]
# print(sum(nums))
```

■ Practice

Practice 1 – Safe Average

```
def safe_average(nums):
    return 0 if not nums else sum(nums)/len(nums)
print(safe_average([]), safe_average([2,4,6]))
```

Practice 2 – Normalize String

```
def normalize(s):
    return ' '.join(s.strip().split())
print(normalize("  hello   world   "))
```

■ Homework

Implement DFS on a tree with parent pointers and compute depth of each node.

```
tree = {1:[2,3], 2:[4,5], 3:[], 4:[], 5:[]}
depth = {}
def dfs(u, d):
    depth[u] = d
    for v in tree[u]:
        dfs(v, d+1)
dfs(1, 0)
print(depth)
```